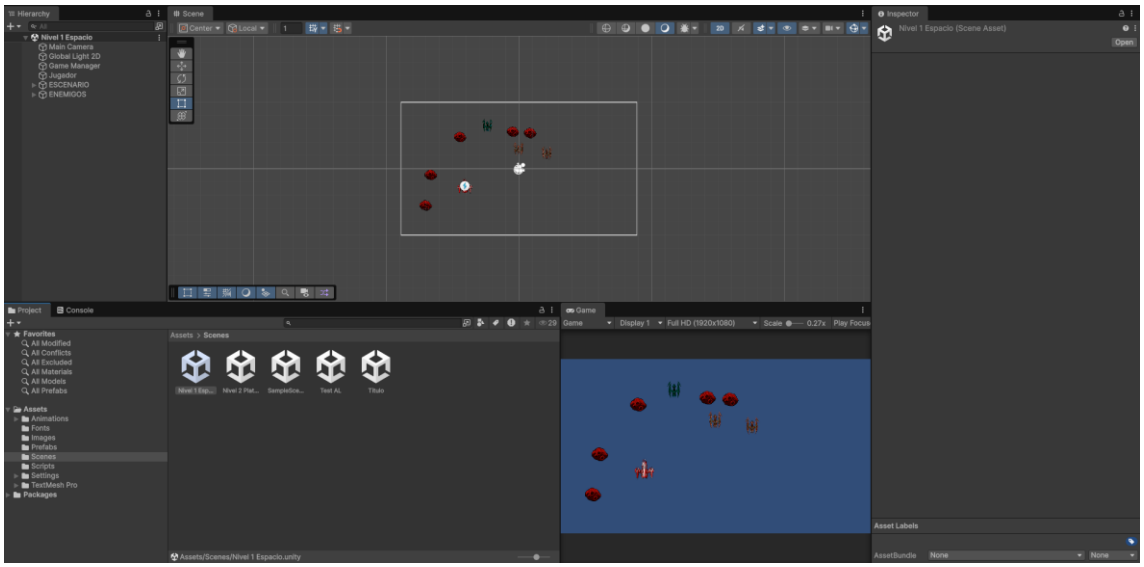
# 1. Entorno Unity

Unity es un motor de desarrollo multiplataforma que permite crear videojuegos en 2D y 3D mediante una combinación de escenas, objetos, componentes, scripts, animaciones, físicas, audio y sistemas de UI. Esta ficha repasa la estructura principal del entorno y los elementos clave que aparecen al abrir el editor.

## 1.1. Interfaz principal de Unity

La ventana principal de Unity se divide en varias áreas diseñadas para cubrir diferentes partes del flujo de trabajo:

- **Scene View**: espacio de trabajo donde se pueden mover, rotar, escalar y colocar GameObjects.
- **Game View**: vista del juego tal como se mostrará al jugador.
- **Hierarchy**: listado jerárquico de todos los GameObjects de la escena.
- **Inspector**: panel donde se editan los componentes de un objeto.
- **Project**: estructura de carpetas con todos los assets del proyecto (sprites, scripts, materiales, prefabs…).
- **Console**: mensajes de debug, errores y advertencias.



## 1.2. Escenas como contenedores de juego

Una **Scene** funciona como un "nivel" o "pantalla". Puede contener:

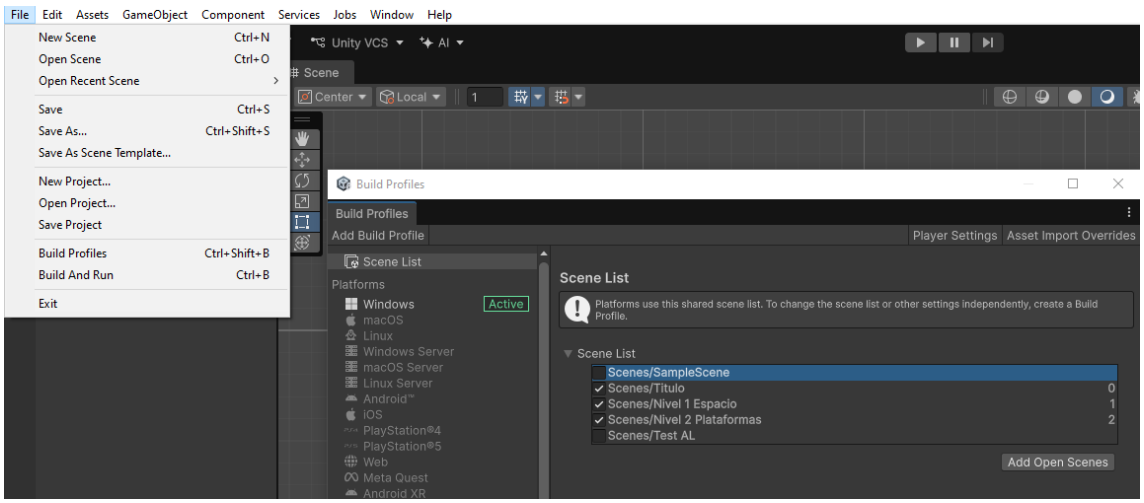
- Jugadores
- Enemigos
- UI
- Suelos y colisiones
- Scripts
- Audio
- Sistemas de partículas

Ejemplo de acceso a escenas desde código:

```
using UnityEngine.SceneManagement;

public class TitleManager : MonoBehaviour
{
    public void Jugar()
    {
        SceneManager.LoadScene(1); // Carga la escena número 1
definida en Build Settings
    }
}
```

La correspondencia entre escenas y su índice se define en:
**File → Build Profiles → Scene List**



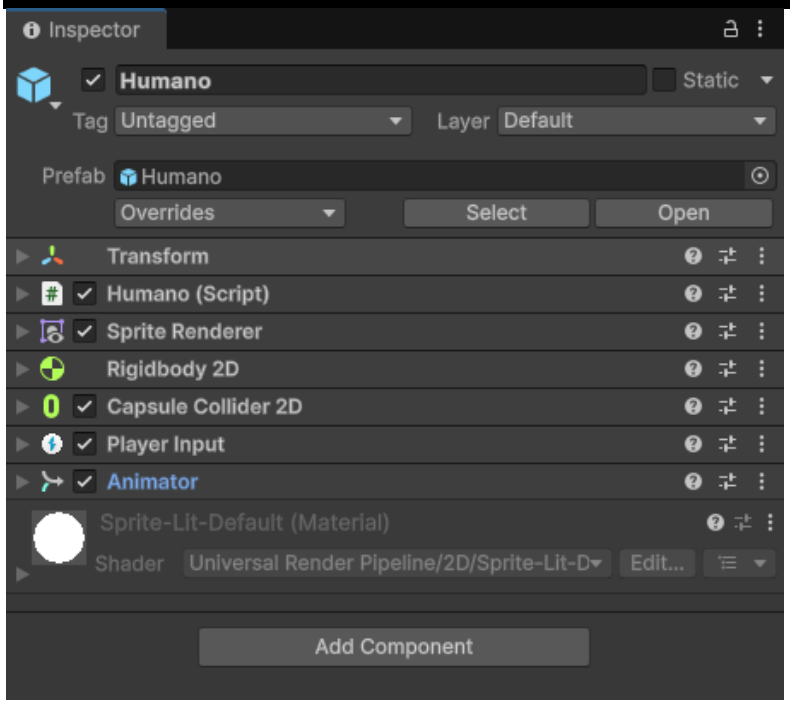
### 1.3. Relación entre GameObjects y componentes

Unity utiliza el paradigma **GameObject + Componentes**.
Un GameObject vacío no tiene comportamiento; la funcionalidad se define añadiendo componentes como:

- Transform
- Sprite Renderer
- Rigidbody2D
- Collider2D
- AudioSource
- Scripts personalizados (por ejemplo, Jugador, Enemigo, Humano…)

Ejemplo en el script **Humano**, donde un GameObject accede a varios componentes:

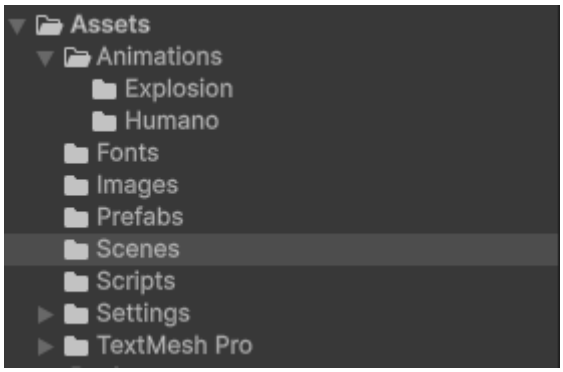
```
Rigidbody2D rb;
Animator animator;
void Start()
{
    rb = GetComponent<Rigidbody2D>();       // Accede al
componente físico
    animator = GetComponent<Animator>();    // Accede al
componente de animación
}
```



### 1.4. Organización del proyecto

En la ventana **Project**, lo habitual es estructurar el proyecto en carpetas como:

- Scripts
- Sprites
- Animations
- Prefabs
- Scenes
- Audio
- Materials
- Particles




Esta organización facilita localizar rápidamente cada recurso y mantiene el proyecto escalable.

### 1.5. Console y debugging

Unity permite imprimir mensajes de depuración mediante print() o Debug.Log(), que aparecen en la ventana **Console**.
Ejemplo real tomado del script **Jugador**:

```
print("Direccion: " + direccion);
print(Time.deltaTime);
```

Estos mensajes ayudan a comprobar valores y detectar errores en tiempo de ejecución.

### 1.6. Ciclo de vida del editor y Play Mode

Unity funciona con dos modos principales:

- **Edit Mode** → se edita la escena.
- **Play Mode** → el juego se ejecuta con física, animación, UI, corrutinas y lógica de scripts.

Cuando se pulsa **Play**, todo método Start, Update, OnEnable, así como corrutinas y físicas, comienzan a ejecutarse.
Ejemplo tomado del script **Enemigo**:

```
void Start()
{
    StartCoroutine(DisparoRepetitivo());
}
```

Una corrutina como esta solo funciona durante el Play Mode.

### 1.7. Ventajas del entorno Unity para programadores

El entorno Unity ofrece:

- Ejecución inmediata del código sin procesos largos de compilación.
- Una API extensa para controlar input, físicas, animación, UI y otras áreas.
- Integración natural con C#.
- Iteración rápida: modificar → probar → ajustar.
- Herramientas visuales que complementan el código, como Animator, Sprite Editor o Inspector.

En el script **Humano**, por ejemplo, se integran varios sistemas del motor:

```
[SerializeField] InputActionReference mover, saltar, atacar;
[SerializeField] TMP_Text textMunicion;
[SerializeField] Image imageBarra;
```

Estos elementos se conectan fácilmente desde el Inspector, aprovechando la interfaz visual del editor.